Computación gráfica y ambientes virtuales (0494)

Práctico 02

Objetivo

El objetivo es dibujar un **cubo indexado unitario** de 24 vértices y 36 índices, con un color plano por cara, usando una estructura de módulos, en el programa, para administrar distintas etapas del plasmado.

Las operaciones de OpenGL para dibujar son las mismas que las aplicadas en el práctico del triángulo lo que se pide ahora es encapsular las operaciones por funcionalidad y por propiedad del recurso (ownership). El resultado de esto debe ser que el `main.cpp` de este práctico no debe poseer llamadas de OpenGL como por ejemplo `glDelete*` (sólo las mínimas necesarias previas al loop y dentro del mismo para dibujar).

Punto de partida

El template de proyecto OpenGL disponible en el aula virtual y el módulo `ResourceManager` del Práctico 01. A esto hay que agregarle los shaders ampliados como se mostraron en las filminas de clase del práctico. En particular, en el *vertex shader* hay que "escribir a mano" la matriz de transformación copiando los valores que están en las filminas. Sin esta transformación el cubo se ve como un cuadrado.

Otra cuestión que hay que usar como "caja negra" es el test de profundidad de OpenGL para que el orden en que se dibujan los triángulos genere una imagen coherente del cubo. Para ello hay que habilitar el test de profundidad con `glEnable(GL_DEPTH_TEST)` antes de iniciar el loop de dibujo y en cada llamada del loop borrar el buffer del test de profundidad con la llamada `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);`.

Definición de los datos de malla

Antes de comenzar a desarrollar los módulos conviene tener definir una interfaz para la **tupla** de datos de cada vértice y como se van a acumular los datos de una malla (indexada o no). A continuación se muestra un ejemplo de una posible definición para estos datos.

```
struct Vertex {                             // Tupla de atributos para un vértice
    float px, py, pz;                       // Posición
    float r,  g,  b;                        // Color
};

struct MeshData {                           // Datos de malla del lado del CPU
    std::vector<Vertex>       vertices;     // Tuplas de cada vértice
    std::vector<unsigned int> indices;      // Índices para graficar los triángulos (opicional)
};
```

Parte 1 — El módulo `Mesh` (administra el VAO, el VBO y el EBO)

Los tres objetos de OpenGL se manejan por *nombres*: enteros que apuntan a memoria que vive en la GPU. Copiar el entero no copia el objeto, y perder el entero no lo libera: lo deja huérfano hasta que se cierra el contexto. De ahí salen las tres decisiones de la interfaz (presentadas en clase).

```
// Mesh.h  --- punto de partida, no molde
class Mesh {
public:
    Mesh() = default;
    ~Mesh();                                // libera lo que posee

    Mesh(const Mesh&) = delete;             // copiar rompe el dueno unico
    Mesh& operator=(const Mesh&) = delete;
    Mesh(Mesh&& other) noexcept;            // mover: se traspasa la propiedad
    Mesh& operator=(Mesh&& other) noexcept;

    void load(const MeshData& data);        // sube a la GPU y arma el VAO
    void clear(void);

    unsigned int vao(void) const  { return vao_; }
    int count(void) const         { return count_; }   // cantidad de INDICES
private:
    unsigned int vao_ {0U}, vbo_ {0U}, ebo_ {0U};
    int count_ {0};
};
```

Funcionalidades que debe tener.

- `load()` crea el VAO, el VBO y el EBO, sube las dos listas y configura los **dos** atributos (posición y color) sobre un único buffer entrelazado.
- El EBO se conecta al VAO por su ranura dedicada.
- El destructor libera. **Ningún `glDelete*` fuera de esta clase.**
- `clear()` se tiene que poder llamar dos veces sin romper nada y sin banderas auxiliares.

Algunas cuestiones a pensar.

- ¿Qué guarda `count_`: vértices o índices? ¿Cuál de los dos necesita `glDrawElements`? ¿Y si se quisiera usar Mesh para dibujar una malla *no indexada*?
- ¿Por qué el objeto *movido* tiene que quedar en cero, y qué pasaría si no se lo pusiera?
- Los desplazamientos de cada atributo dentro de la tupla, ¿se escriben a mano (0 y 12) o se calculan? ¿Qué pasa el día que la tupla lleve la normal en el medio?
- Si mañana el vértice tuviera un atributo más, ¿cuántos lugares del código habría que tocar?

Parte 2 — El módulo `Shader` (administra el programa linkeado)

Es la misma estrategia planteada para el módulo `Mesh` sobre otro recurso: un programa de `shader` que se identifica por un *nombre* (un único entero en vez de tres). Lo que cambia es el tipo de recurso que se administra.

```
// Shader.h  --- punto de partida, no molde
class Shader {
public:
    Shader() = default;
    ~Shader();                                  // libera el programa
    Shader(const Shader&) = delete;             // las MISMAS tres decisiones
    Shader& operator=(const Shader&) = delete;
    Shader(Shader&& other) noexcept;
    Shader& operator=(Shader&& other) noexcept;

    bool compile_from_source(const std::string& vs, const std::string& fs);
    void use(void) const;
    void clear(void);

    unsigned int id(void) const  { return id_; }
private:
    unsigned int id_ {0U};
};
```

Funcionalidades que debe tener.

- Compila los dos componentes, los linkea y libera los objetos intermedios.
- **Esta clase no abre archivos.** Recibe el código fuente ya leído: quien lee del disco es el `ResourceManager` del Práctico 01.
- Si algo no compila o no linkea, como mínimo **se pide y se imprime el log del driver**, y el objeto queda vacío. Un shader roto no puede pasar desapercibido: es la falla 2 de la clínica del triángulo.

Algunas cuestiones a pensar.

- El largo del log, ¿se asume fijo con `char[512]` o se consulta?
- ¿Cómo se avisa el error: valor de retorno, excepción, otra cosa? ¿Es coherente con lo que decidieron en el `ResourceManager`?

Parte 3 — El módulo `primitives` (generador de mallas de figuras primitivas)

Los 24 vértices no se escriben a mano en el `main`: se obtienen por medio de una función. Este módulo hoy tiene una sola primitiva, pero en el próximo práctico se le agregaran el cono y el cilindro; y en el futuro se le pueden continuar integrando otras primitivas que hagan falta (ej. disco, quad, triángulo, etc.).

```
// Primitives.h  --- punto de partida, no molde
namespace primitives {

// 24 vertices, 36 indices, color por cara
MeshData cube(float scale_x = 1.0f, float scale_y = 1.0f, float scale_z = 1.0f);

}  // namespace primitives
```

Funcionalidades que debe tener.

- La función `cube(...)` devuelve la malla **indexada**: 24 vértices únicos y 36 índices, con un color plano distinto por cara (esto está escrito a "mano" dentro de la función).
- No usa la API de OpenGL: es un generador de *datos*.
- Las cuatro esquinas de cada cara se recorren siempre en el mismo sentido (antihorario mirando la cara desde afuera). Conviene fijar este criterio ahora y no cambiarlo (de esto depende el descarte de caras traseras que se verá más adelante en el teórico y decide que cara se dibuja y cuál no).
- Los argumentos de la función permiten cambiar el tamaño del cubo en las tres direcciones (X,Y,Z) con valores arbitrarios por lo que se podrían obtener otras figuras de caras rectas paralelas (ej. una placa). Con el valor por defecto devuelven un cubo de lado unitario ($l = 1$).

Algunas cuestiones a pensar.

- ¿Función libre o clase? La pregunta de la clase del práctico — *¿qué posee este objeto?* — acá tiene una respuesta clara, y la respuesta decide la forma.
- ¿Los colores son un parámetro o quedan fijos adentro del generador?
- El cubo, ¿queda centrado en el origen o apoyado sobre el plano `y = 0`?
- Pensando en la ampliación del módulo: ¿esta firma sirve tal cual para `cylinder(radio, altura, gajos)` y `cone(...)`, o hay que reescribir el módulo cuando aparezcan? Si hay que reescribirlo, todavía se está a tiempo de cambiar la forma.

Parte 4 — El programa que los usa

Armar el `main.cpp` sobre el template, con las cuatro etapas del triángulo: decidir los datos, pasarlos a la GPU, armar el programa de shaders, dibujar. Las tres primeras etapas se deberían encargar los módulos mientras que la función para dibujar ahora es `glDrawElements`.

Recomendaciones.

1. El template trae un solo fuente. Hay que ajustar el sistema de compilación para compilar varios archivos. SI se utiliza el Makefile provisto sólo hay que agregar el subdirectorio de los módulos a la variable `LIB_DIRS` del `Makefile` (el mecanismo está documentado en el archivo).
2. **Los objetos que poseen recursos de OpenGL tienen que morir con el contexto todavía vivo**, o sea antes de que se cierre GLFW. Si viven hasta el final de `main`, sus destructores corren sin contexto: a veces no pasa nada y a veces es un *segfault* al cerrar. Para este práctico un bloque `{ }` alrededor de los objetos alcanza.
3. Que compile sin ningún warning
4. Documentar mínimamente en cada interfaz las decisiones tomadas.

Cómo se sabe que está bien.

- Se ven **tres caras** de tres colores distintos, sin caras que aparezcan y desaparezcan.
- El programa puede hacer una salida a consola indicando cuántos vértices y cuántos índices generó: tienen que ser **24 y 36**.
- No hay un solo `glDelete*` en el `main.cpp`.
- **Probar a romperlo**, como en la clínica: dibujar con `glDrawArrays` en vez de `glDrawElements`, con el mismo número — 36 — y sin tocar nada más. Predecir por escrito qué se va a ver *antes* de correrlo. Ninguna de estas fallas produce un error de OpenGL: hay que visualizarlo ó predecirlo.

Como en el Práctico 01, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno, y hay que poder explicarlas.